CSC 2201 · EXAM GUIDE

# Programming from zero

Everything from the course explained in plain language, for someone who has never written a line of code. With a cheat sheet and a quiz.



Covers all lectures from 18.09.2026 to 26.10.2026 and the full exercise sheet · Practise online: brtzl.github.io/study-shelf

# What’s inside


1 **Thinking in steps** 4
Algorithms, the omelette, the three building blocks

2 **Flowcharts** 6
The six symbols and the lecturer’s notation

3 **Your first Python program** 9
Variables, types, input and print, operators

4 **Making decisions** 14
if, elif, else, and/or/not

5 **Repeating things** 18
for, range, while, break, continue

6 **Functions** 23
def, return, and reusing code

7 **The classic algorithms** 26
GCD, Euclid, Fibonacci, factorial, digits, binary

8 **Exam question types** 36
A method for each, plus a final checklist

9 **The exercise sheet, solved** 39
All 16 exercises with real output

10 **Cheat sheet** 52
Two pages to print and review

11 **Quiz** 54
44 multiple choice and 7 written questions

12 **Answers** 63
With a short explanation for every question


**Spelling**

The course materials sometimes write “Phyton” and “divident”. The correct spellings are **Python** and **dividend**. “49 VE ALTI” in the original grade table is Turkish for “49 and below”.

# How to use this guide

You don’t need any background. Read in order, because each chapter builds on the one before.

| Colored box | What it tells you |
|---|---|
| ! Red | A trap. Where students commonly lose points, and what the lecturer warned about. |
| ★ Yellow | A useful tip or memory trick. |
| → Teal | A big idea worth pausing on. |
| ✓ Purple | Something to learn word for word. |
| i Grey | A side note, including places where the course material has a small mistake. |

## A code box looks like this

```
1  name = "Python"
2  print(f"Hello {name}")   # a note for humans
```

ON THE SCREEN

```
Hello Python
```

The light part is the program. The dark part, “On the screen”, is what appears when it runs. Line numbers are not part of the code. Every program in this guide has been run, so the output is real.

## A plan if you have a week

| Day | Read | Then do |
|---|---|---|
| 1 | Chapters 1–2 | Draw the flowchart for making tea. |
| 2 | Chapter 3 | Quiz: the Basics questions. |
| 3 | Chapter 4 | Write the grade program without looking. |
| 4 | Chapter 5 | Trace every code box by hand. |
| 5 | Chapters 6–7 | Pick 3 classics and write them from memory. |
| 6 | Chapters 8–9 | Solve the exercises, then compare. |
| 7 | Chapters 10–11 | Take the full quiz. Review what you got wrong. |

**Practise on paper**

Questions like the ones on the exercise sheet are answered on paper, so practise with a pen. The companion website, **https://brtzl.github.io/study-shelf/**, has the quiz with instant feedback, flashcards, and a player that walks through programs line by line.

CHAPTER 1

# Thinking in steps

Before any code: what a computer actually is, and why a recipe is the best way to think about programming.

## The omelette

Your course begins with a conversation in a kitchen. Someone says “I am hungry. Let’s make an omelette.” The other person has never made one. So the first person talks them through it:

**Step 1.** Prepare the ingredients on the counter: 3 eggs, sunflower oil, butter, salt, black pepper. We need a pan.

**Step 2.** Put the pan on the stove and turn the heat to medium.

**Step 3.** Put 2 spoons of sunflower oil and a spoon of butter into the pan.

**Step 4.** Crack the eggs into the glass bowl.

**Step 5.** Beat the eggs.

**Step 6.** Add a little salt and black pepper, and whisk a bit more.

**Step 7.** Pour the mixture into the pan and let it cook.

**Step 8.** You can serve it now.

That list is an **algorithm**. A program is the same thing, written for a computer. Think of the computer as a very fast helper who has never cooked anything, and who does *exactly* what you say and nothing more.

## A computer needs details

The lecture tells a second story. You ask a helper to add numbers: “Take numbers from me, calculate the sum, show the result.” It works. 14 and 35 gives 49.

Then you ask for a division in the same way, and the helper says “problem”. Adding works in any order, but dividing does not: 21 ÷ 3 is not 3 ÷ 21. So you must be specific: “Take the **dividend** and the **divisor**. Calculate dividend / divisor.” Now 21 and 3 gives 7.

→ **The big lesson**

A computer never guesses what you meant. If a step could be understood in two ways, it is not good enough. This is the “definiteness” rule below.

## What is an algorithm?

**Learn this definition word for word**

**An algorithm is a method of solution that is expressed as a sequence of finite steps.**

Every algorithm has four characteristics:

| Characteristic | What the slides say | In plain words |
|---|---|---|
| **Input** | An algorithm may or may not require input. | It might need ingredients from you. It might not. |
| **Output** | Each algorithm is expected to produce at least one result. | There must be an omelette at the end. |
| **Definiteness** | Each instruction must be clear and unambiguous. | "Add a bit of something" is not allowed. |
| **Finiteness** | If the instructions are executed, the algorithm should terminate after a finite number of steps. | It must end. It cannot cook forever. |

## The three building blocks

Algorithms and flowcharts are built from only three **control structures**. Every program in this course is a mix of these:

| Structure | Meaning | Everyday example |
|---|---|---|
| **1. Sequence** | Do this, then this, then this. | Crack the eggs, then beat them. |
| **2. Branching (Selection)** | Choose one path based on a question. | If it is raining, take an umbrella. Otherwise, take sunglasses. |
| **3. Loop (Repetition)** | Repeat some steps. | Keep whisking until the mixture is smooth. |

CHAPTER 2

# Flowcharts: an algorithm as a drawing

A flowchart shows the same steps as an algorithm, using shapes and arrows. Exam questions ask you to draw them and to turn them into code.

## What is a flowchart?

- The first flowchart design goes back to **1945** and, according to the slides, was designed by **John von Neumann**.
- Unlike an algorithm written in words, a flowchart uses **different symbols** to design a solution.
- By looking at one, you can understand the operations and the order they happen in.
- A flowchart is often considered a **blueprint** of a design used to solve a specific problem.

A free tool for drawing them is `app.diagrams.net`.

## The six symbols

| Name | Symbol | What it is for | Example |
|---|---|---|---|
| **Oval** |  | The start and the end of the flowchart. | Start, Stop |
| **Parallelogram** |  | Input and output. | TAKE a, SHOW result |
| **Diamond** |  | A decision. There are alternatives, usually true and false. | a > b ? |
| **Rectangle** |  | Processing: calculations and changing data. | sum ← a + b |
| **Arrows** |  | Flow lines. They connect the symbols and show the order. | → |
| **Predefined process** |  | A function: a group of steps doing one task, defined elsewhere. | w = f(c) |

**Memory trick**

**Parallelogram** = the computer talks with a person (TAKE, SHOW). **Rectangle** = the computer thinks alone. **Diamond** = a question with two exits.

## How your lecturer writes things

Exams say “write an algorithm” and “draw the flowchart”, so copy this style:

| Notation | Meaning |
|---|---|
| `Step 1.` `Step 2.` … | Numbered steps, in order. |
| `Step 3.1` | A step that sits inside step 3, such as the “then” part of an “if”. |
| `sum ← a + b` | The left arrow means “store the result in”. Calculate the right side, then put it in `sum`. |
| `if … then … else …` | A branch: a true part and a false part. |

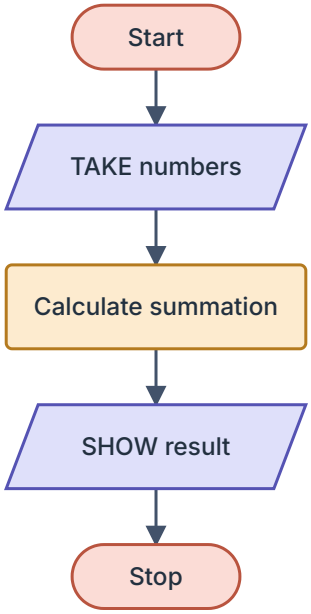
| Notation | Meaning |
|---|---|
| `go to step 3` | Jump back. This creates a loop. |
| `TAKE`, `SHOW` | Input and output. |
| `i \| N` | “i divides N” with nothing left over. 3 \| 12 is true. 5 \| 12 is false. |
| `N mod 10` | The remainder after dividing. 851 mod 10 = 1. |
| small empty circle | A merge point, where several arrows join into one. |

### Example: summing numbers

**Step 1.** Take numbers.
**Step 2.** Calculate summation.
**Step 3.** Show result.

Dividing looks the same, but the input is named exactly:

**Step 1.** Take dividend and divisor.
**Step 2.** Calculate division as dividend / divisor.
**Step 3.** Show result.




A sequence: top to bottom, no choices.

**You can combine**
Two processes can share one rectangle, as long as you write them in the correct order.

**Order matters**
“a ← b” then “b ← a” does not swap anything. By the second step, the old value of a is gone. There is more on this in Chapter 3.

CHAPTER 3

# Your first Python program

Python is the language we write these steps in. This chapter covers the vocabulary: storing values, asking the user, doing maths, showing answers.

## Reading a whole program first

Here is the volume of a cylindrical tank, V = π r² h. Read it like a recipe. The computer goes through the lines from top to bottom, one at a time.

```
1  r = int(input("please enter the radius : "))
2  h = int(input("please enter the height : "))
3  V = 3.14159 * r * r * h
4  print(f"The volume is {V}")
```

ON THE SCREEN

```
please enter the radius : 5
please enter the height : 2
The volume is 157.0795
```

| Line | What it means in English | Algorithm step |
|---|---|---|
| 1–2 | Ask for the radius and height, and remember them as `r` and `h`. | Step 1. Take r and h |
| 3 | Calculate the volume and remember it as `V`. | Step 2. V ← π·r·r·h |
| 4 | Show the answer on the screen. | Step 3. Show V |

## Variables: labelled boxes

A **variable** is a box with a name on it, holding one value. The line `age = 20` means “put 20 in the box called age”. If you put in something new, the old value is thrown away.

**= does not mean “equals” here**

`x = x + 1` looks impossible in maths. In Python, `=` means “store”. It works out the right side using the old x, then stores the answer back in x. Your lecturer’s arrow says it better: `x ← x + 1`. To *ask* whether two things are equal, use **two** equals signs: `==`.

### Naming rules

- Must start with a **letter or underscore**.
- Cannot start with a **number**.
- Only letters, digits and underscores (A–z, 0–9, _).
- **Case-sensitive**: `age`, `Age` and `AGE` are three different variables.
- Cannot be a Python keyword, such as `not`.

| Name | OK? |
|---|---|
| `my_name`, `_my_name` | ✓ |
| `midterm1`, `midterm_1` | ✓ |
| `e_posta_adresi` | ✓ |
| `my-name` (has a dash) | ✗ |
| `age!of!you` (has !) | ✗ |
| `1st_midterm` (starts with a digit) | ✗ |
| `not` (keyword) | ✗ |

> **A small detail**
> The lecture notes also list `print`, `int` and `input` as names not to use. Strictly, Python would allow them, but you would break those tools for the rest of your program. In the exam, treat them as forbidden.

## Data types: kinds of values

| Type | Short for | Holds | Examples |
|---|---|---|---|
| `int` | integer | whole numbers | `7`, `-3`, `2026` |
| `float` | floating point | decimal numbers | `3.14`, `15.0` |
| `str` | string | text, in quotes | `"volkan"`, `"7"` |
| `bool` | boolean | only True or False | `True`, `False` |
| `complex` | complex number | numbers with an imaginary part | `1j` is *i* in Python |

The slides also list these other built-in types by name. You only need to recognise that they exist: `list`, `tuple`, `range`, `dict`, `set`, `frozenset`, `bytes`, `bytearray`, `memoryview`, `NoneType`.

## Talking to the user

`input("question")` shows the question and waits for typing. What it gives back is always **text**, even if a number was typed. Text can't be used in maths, so we convert it:

| You write | You get | Use it for |
|---|---|---|
| `input("…")` | text | words |
| `int(input("…"))` | a whole number | counts, integers |
| `float(input("…"))` | a decimal number | measurements, anything with a point |

`print(…)` shows something on the screen. There are two common ways to mix text with values:

```python
summ = 15
print("sum of your numbers is: ", summ)   # comma: print adds a space
print(f"sum of your numbers is: {summ}")  # f-string: {summ} shows its value
print("A\tB")                             # \t is one tab (about 4 spaces)
print("no new line", end="")              # end="" means: stay on the same line
print(" ...still the same line")
```

ON THE SCREEN

```
sum of your numbers is:  15
sum of your numbers is: 15
A       B
no new line ...still the same line
```

**Comments**

Anything after `#` is a note for humans. The computer ignores it.

## Maths in Python

| Symbol | Meaning | Example | Result |
|---|---|---|---|
| `+` `-` | add, subtract | `7 - 2` | 5 |
| `*` | multiply | `3 * 4` | 12 |
| `/` | divide | `851 / 10` | 85.1 |
| `//` | integer part of a division | `851 // 10` | 85 |
| `%` | **remainder** (mod) | `851 % 10` | 1 |
| `**` | power | `2 ** 3` | 8 |

**Never write r^2**

The lecturer's warning: "Do NOT use r^2!" In Python, `^` is a different operation, and it will silently give you a wrong number. For powers use `r ** 2` or `m.pow(r, 2)`.

**Division always gives a decimal**

If an expression contains `/` or at least one float, the result is a float. `10 / 2` is `5.0`, not 5. With whole numbers, `//` gives a whole number, or you can wrap a result: `int(25.5)` gives `25`. Careful with negatives: `//` always rounds *down*, so `-7 // 3` is `-3`, while `int(-7 / 3)` is `-2`.

**Parentheses change the answer**

Python multiplies and divides before it adds, going left to right.
`-b + m.sqrt(delta) / (2*a)` only divides the square root.
`(-b + m.sqrt(delta)) / 2*a` divides by 2 and then *multiplies* by a. The correct form is `(-b + m.sqrt(delta)) / (2*a)`.

### The remainder is a superpower

| Question | Python | Why |
|---|---|---|
| Is n even? | `n % 2 == 0` | Dividing by 2 leaves nothing. |
| Does 5 divide n? | `n % 5 == 0` | Dividing by 5 leaves nothing. |
| Last digit of n? | `n % 10` | 851 % 10 is 1. |
| Remove the last digit? | `n // 10` | 851 // 10 is 85. |

### Shortcuts

| Shortcut | Same as | Note |
|---|---|---|
| `i += 1` | `i = i + 1` | The most common line in this course |
| `n -= d` | `n = n - d` | |
| `f *= i` | `f = f * i` | |
| `x /= 2` | `x = x / 2` | |
| `f1 = f2 = 1` | both become 1 | |
| `s, B = N2, N1` | `s = N2` and `B = N1` at the same moment | Swapping: `a, b = b, a` |

## Extra tools: import

Python keeps extra tools in **packages**. `import math as m` brings in the maths package and gives it the short nickname (alias) `m`. Other common ones are `sympy as sp` and `numpy as np`.

m.pi is π, m.pow(a, b) is a to the power b, m.sqrt(x) is a square root

```python
import math as m
r = int(input("please enter the radius : "))
h = int(input("please enter the height : "))
V = m.pi * m.pow(r, 2) * h
print(f"The volume is {V}")
print(m.sqrt(16), 2 ** 2.15 == m.pow(2, 2.15))
```

ON THE SCREEN

```
please enter the radius : 5
please enter the height : 2
The volume is 157.07963267948966
4.0 True
```

**Order again**

A value must exist before it is used. If you write `pi = 3.14159`, it must come **before** the line that calculates V.

CHAPTER 4

# Making decisions: if, elif, else

The second building block, branching. The program asks a yes-or-no question and takes a different path depending on the answer.

## True-or-false questions

A **condition** (also called a logical expression) is a question whose answer is `True` or `False`.

| Write | Means | Example that is True |
|---|---|---|
| `==` | equal? | `3 == 3` |
| `!=` | not equal? | `3 != 4` |
| `<` `>` | less than, greater than | `2 < 5` |
| `<=` `>=` | less or equal, greater or equal | `5 >= 5` |

You can join questions together:

| Word | True when | Example |
|---|---|---|
| `and` | **both** sides are true | `N1 % i == 0 and N2 % i == 0` |
| `or` | **at least one** side is true | `N == 0 or N == 1` |
| `not` | flips the answer | `not (N == 0 or N == 1)` is the same as `N != 0 and N != 1` |

## The if statement

The pattern

```
if condition:
    # true part
    ...
else:
    # false part
    ...
```

An example

```
a = 7
if a % 2 == 0:
    print("even")
else:
    print("odd")
print("done")
```

ON THE SCREEN

```
odd
done
```

**Colon and indentation**

The line ends with a **colon** `:`. The lines that belong inside are pushed to the right. This is called **indentation**, and it is not just decoration. It is how Python knows which lines are inside the if. `print("done")` above is not indented, so it runs either way.

## Worked example: roots of a quadratic

For a polynomial $ax^2 + bx + c$, the roots depend on delta ($\Delta = b^2 - 4ac$). If delta is negative, the square root needs the imaginary number *i*. You don't need the maths behind it. Just see how the choice works. The value of **a must not be 0**: without an $x^2$ term this isn't a quadratic, and the code would divide by zero.

**Step 1.** Take a (coefficient of $x^2$), b (coefficient of x), c (constant).

**Step 2.** delta ← $b^2 - 4ac$

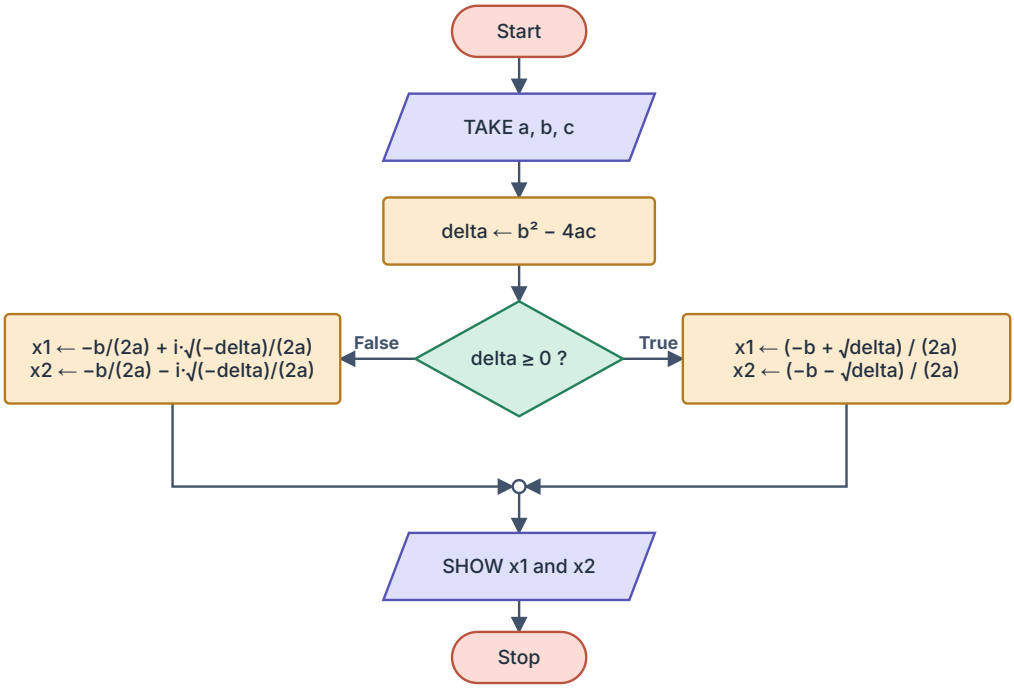
**Step 3.** if delta ≥ 0 then

**Step 3.1** x1 ← (−b + √delta) / (2a), x2 ← (−b − √delta) / (2a)

else

**Step 3.1** x1 ← −b/(2a) + i·√(−delta)/(2a), x2 ← −b/(2a) − i·√(−delta)/(2a)

**Step 4.** Show x1 and x2.




Branching: both paths meet again at the small circle, the "merge" dot.

```python
import math as m

a = float(input("enter the coefficient of x^2 : "))
b = float(input("enter the coefficient of x : "))
c = float(input("enter the constant : "))

delta = b ** 2 - 4 * a * c

if delta >= 0:
    x1 = (-b + m.sqrt(delta)) / (2 * a)
    x2 = (-b - m.sqrt(delta)) / (2 * a)
else:
    r = -b / (2 * a)
    s = m.sqrt(-delta) / (2 * a)
    x1 = r + 1j * s
    x2 = r - 1j * s

print(f"x1 : {x1}\tx2 : {x2}")
```

ON THE SCREEN

```
enter the coefficient of x^2 : 1
enter the coefficient of x : 2
enter the constant : 5
x1 : (-1+2j)    x2 : (-1-2j)
```

Tested with a=1, b=2, c=5. `1j` is how Python writes *i*.

## More than two choices: elif

`elif` means "if the previous conditions were not true, then try this one". Python checks from the top and **stops at the first true one**. The rest are skipped.

```python
a = 33
b = 33
if b > a:
    print("b is greater than a")
elif a == b:
    print("a and b are equal")
```

ON THE SCREEN

```
a and b are equal
```

### Worked example: the final grade

Avg = midterm × 0.3 + quiz × 0.2 + final × 0.5. Then use this table:

| Avg | 90–100 | 85–89 | 80–84 | 75–79 | 70–74 | 65–69 | 60–64 | 50–59 | 49 and below |
|---|---|---|---|---|---|---|---|---|---|
| **Grade** | AA | BA | BB | CB | CC | DC | DD | FD | FF |
| **Point** | 4.00 | 3.50 | 3.00 | 2.50 | 2.00 | 1.50 | 1.00 | 0.50 | 0.00 |

```python
m = int(input("enter midterm : "))
q = int(input("enter quiz : "))
f = int(input("enter final : "))

avg = m * 0.3 + q * 0.2 + f * 0.5

if avg >= 90:
    grade = "AA"
elif avg >= 85:
    grade = "BA"
elif avg >= 80:
    grade = "BB"
elif avg >= 75:
    grade = "CB"
elif avg >= 70:
    grade = "CC"
elif avg >= 65:
    grade = "DC"
elif avg >= 60:
    grade = "DD"
elif avg >= 50:
    grade = "FD"
else:
    grade = "FF"

print(grade)
```

ON THE SCREEN

```
enter midterm : 80
enter quiz : 70
enter final : 90
BB
```

→ **Why only >= is needed**

When the program reaches `avg >= 85`, it already knows avg is below 90, or the first line would have caught it. So there is no need to write "between 85 and 89". This only works if you check from the **highest down**.

i **A mismatch in the notes**

The grade table says FD is 50–59, but the code in the notes has `elif avg >= 55` for FD. We use 50 here because it matches the table. If this comes up in the exam, follow the table in the question.

★ **The last possibility**

The final choice needs no question. If nothing above was true, `else` catches everything left over. The flowchart shows this too: the last box, FF, has no diamond.

CHAPTER 5

# Repeating things: loops

The third building block. Loops are where most exam points are, so take this chapter slowly.

## Why we need loops

To add 100 numbers, we could write 100 “take a number” steps. The lecturer’s better idea: keep a running total, like a shopping receipt.

| Number typed | – | 1 | 3 | 10 |
|---|---|---|---|---|
| sum (running total) | 0 | 1 | 4 | 14 |

**Step 1.** Take N (how many numbers).
**Step 2.** sum ← 0, c ← 0 (c is a **counter**)
**Step 3.** Take a number n.
**Step 4.** sum ← sum + n
**Step 5.** c ← c + 1
**Step 6.** if c < N then go to step 3
**Step 7.** Show sum.

**The two loop patterns**

**Counter:** starts at 0, goes up by 1 each round. It answers “how many?”.

**Accumulator:** starts at 0 and adds a value each round. It answers “what is the total?”.

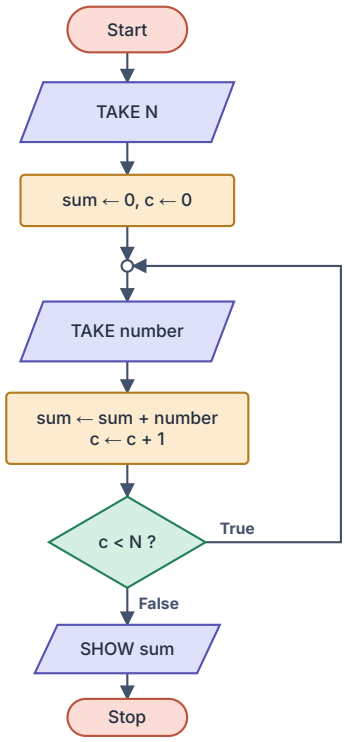
For a running *product*, like a factorial, start at **1**, because anything times 0 is 0.

## The for loop

`for` takes the items of a group one at a time. Each round is called an **iteration**.

```
1  for t in "volkan":
2      print(t)
3  print("have a nice days...")
```

```
ON THE SCREEN
v
o
l
k
a
n
have a nice days...
```




A loop: the arrow going back up is what makes it repeat.

| Part | Meaning |
|---|---|
| `t` | A variable that holds the current item. You choose the name. |
| `"volkan"` | The iterable object: anything that can be gone through item by item. A string gives one letter at a time. |
| Indented lines | The **scope** of the for. These lines repeat. The unindented `print` runs once, afterwards. |

## range(): making a list of numbers to count through

| You write | Gives | Rule |
|---|---|---|
| `range(6)` | 0, 1, 2, 3, 4, 5 | Starts at 0. 6 items, ending at 6−1. |
| `range(2, 6)` | 2, 3, 4, 5 | From start, up to but **not including** the end. |
| `range(2, 22, 5)` | 2, 7, 12, 17 | Third number is the step size. |
| `range(5, 0, -1)` | 5, 4, 3, 2, 1 | A negative step counts down. |
| `range(2, S+1)` | 2, 3, ..., S | To include S, end at S+1. |

**The end is never included**

`range(1, 5)` stops at 4. This is the single most common mistake. To include N, write `N + 1`. To count down to 1, end at 0.

The summing algorithm in Python

```
1  N = int(input("how many numbers do you have? "))
2  summ = 0
3
4  for i in range(N):
5      number = float(input("enter the number: "))
6      summ = summ + number
7
8  print("sum of your numbers is: ", summ)
```

ON THE SCREEN

```
how many numbers do you have? 2
enter the number: 10
enter the number: 5
sum of your numbers is:  15.0
```

**Why 15.0 and not 15?**

Because of `float(...)`: the numbers are decimals, so the total is a decimal too.

## A loop inside a loop

The **inner** loop runs all its rounds for **each single round** of the **outer** loop, like a clock: the minute hand makes a full circle for each step of the hour hand. Indentation shows which line belongs to which loop.

i stays at 1 while j goes 1, 2, 3. Then i becomes 2 and j starts again

```python
for i in range(1, 3):
    for j in range(1, 4):
        print(i, j)
```

ON THE SCREEN

```
1 1
1 2
1 3
2 1
2 2
2 3
```

## The while loop

`while` repeats **as long as a condition is true**. Use it when you don't know in advance how many rounds you need.

```python
i = 1
while i <= 3:
    print(i)
    i += 1

print("have a nice day...")
```

ON THE SCREEN

```
1
2
3
have a nice day...
```

**The infinite loop**

Lecturer's note: "be sure, after a while, condition must be FALSE." If you forget `i += 1`, then i stays 1, the condition stays true, and the loop never ends. A `while` needs three things: **prepare** the counter before the loop, **test** it in the condition, and **change** it inside.

## break and continue

| Word | What it does | In the kitchen |
|---|---|---|
| `break` | Leave the loop right now. | The omelette burned. Stop cooking. |
| `continue` | Skip the rest of this round and start the next. | This egg is bad. Throw it away and take the next one. |

```python
for i in range(5):
    if i == 2:
        continue
    print(i)
print("have a nice day...")
```

ON THE SCREEN

```
0
1
3
4
have a nice day...
```

```python
for i in range(5):
    if i == 2:
        break
    print(i)
print("have a nice day...")
```

ON THE SCREEN

```
0
1
have a nice day...
```

### while True: the “loop with a check in the middle”

`while True:` would run forever, so inside there must be a `break`. This is the easy way to turn a flowchart into code when its diamond is at the bottom or the middle.

Add up positive numbers, ignore negatives, stop at zero

```python
s = 0
while True:
    number = int(input("Enter a number: "))
    if number < 0:
        continue
    if number == 0:
        print(f"sum of your positive numbers is {s}")
        break
    s += number

print("have a nice day...")
```

ON THE SCREEN

```
Enter a number: 7
Enter a number: -1
Enter a number: 3
Enter a number: 0
sum of your positive numbers is 10
have a nice day...
```

| | for | while |
|---|---|---|
| **Use when** | you know how many rounds, or have a group to go through | you repeat until something happens |
| **Counter** | handled for you | you prepare it and change it yourself |
| **Example** | add N numbers; factorial | Euclid’s GCD; sum of digits |

CHAPTER 6

# Functions: a recipe you can reuse

When a task appears several times, give it a name once, then just call it by name.

## The idea

The slides say: a function is a **block of code which only runs when it is called**. A function **can** return data as a result. A function helps to **avoid code repetition**. Another word for it is a **method**.

Think of a "Make Tea" card. Writing the card doesn't make any tea. Nothing happens until someone says "make tea". Then they follow the card and hand you a cup.

| Part | Meaning | Tea card |
|---|---|---|
| `def` | "I am defining a function." | Writing the card |
| `factorial` | The name you choose. | "Make Tea" |
| `(n)` | **Parameters**: the values it needs to calculate the result. | "how many cups?" |
| indented lines | The **scope** of the function: its steps. | The steps on the card |
| `return f` | Hand the result back to whoever asked. | Handing you the cup |

```
1  def factorial(n):
2      f = 1
3      for i in range(1, n + 1):
4          f *= i
5      return f
6
7
8  n = int(input("enter an integer: "))
9  print(factorial(n))
```

ON THE SCREEN

```
enter an integer: 7
5040
```

The computer skips over lines 1–5 at first, since they are only the definition. It starts on line 8. On line 9 we **call** `factorial(n)`. It jumps up with the given number, calculates, and `return` brings the result back to where the call was.

## Why it helps: the binomial coefficient

The formula C(n, r) = n! / ((n−r)! · r!) needs a factorial **3 times**. Without a function we would write the same loop three times.

```python
def factorial(n):
    f = 1
    for i in range(1, n + 1):
        f *= i
    return f


def c(u, v):
    return factorial(u) / (factorial(v) * factorial(u - v))


n = int(input("enter n: "))
r = int(input("enter r: "))
b = c(n, r)
print(f"binomial({n},{r}) = {b}")
```

ON THE SCREEN

```
enter n: 5
enter r: 3
binomial(5,3) = 10.0
```

**Functions can call functions**

`c` calls `factorial`. The parameter names (`u, v`) do not need to match the names used at the call (`n, r`). Values are passed by position: n goes to u, r goes to v.

**In a flowchart**

A function call is drawn with the double-sided box, like `w = f(c)`.

## Worked example: a long sum

Calculate 1 + x/1! + $x^2$/2! + $x^3$/3! + … + $x^M$/M!, with M = 50 and x given by the user. This combines two older ideas: **summing with a loop** (02.10) and **factorial as a function** (23.10).

```python
def factorial(n):
    f = 1
    for i in range(2, n + 1):
        f *= i
    return f


M = 50
x = float(input("Enter x : "))
summ = 1

for k in range(1, M + 1):
    ak = x ** k / factorial(k)
    summ += ak

print(f"sum is {summ}")
```

ON THE SCREEN

```
Enter x : 1
sum is 2.7182818284590455
```

`summ` starts at 1 because the first term, $x^0/0!$, equals 1. The answer for x = 1 is the famous number *e*.

### Stopping when terms are too small to matter

Instead of a fixed 50 terms, we can stop as soon as a term is tinier than a chosen limit (`eps`, $10^{-15}$). Later terms are even smaller, so we don't need them. We don't know the number of rounds in advance, so we use `while`.

```python
def factorial(n):
    f = 1
    for i in range(2, n + 1):
        f *= i
    return f


eps = 10 ** (-15)
x = float(input("Enter x : "))
summ = 1
k = 1
while True:
    ak = x ** k / factorial(k)
    summ += ak
    if abs(ak) < eps:
        break
    k += 1

print(f"sum is {summ}, with {k+1} terms.")
```

ON THE SCREEN

```
Enter x : 1
sum is 2.7182818284590455, with 19 terms.
```

`abs(...)` removes a minus sign: abs(−3) is 3.

CHAPTER 7

# The classic algorithms

Every algorithm taught in the lectures, in the order they were taught: first the idea by hand, then the steps, then the Python.

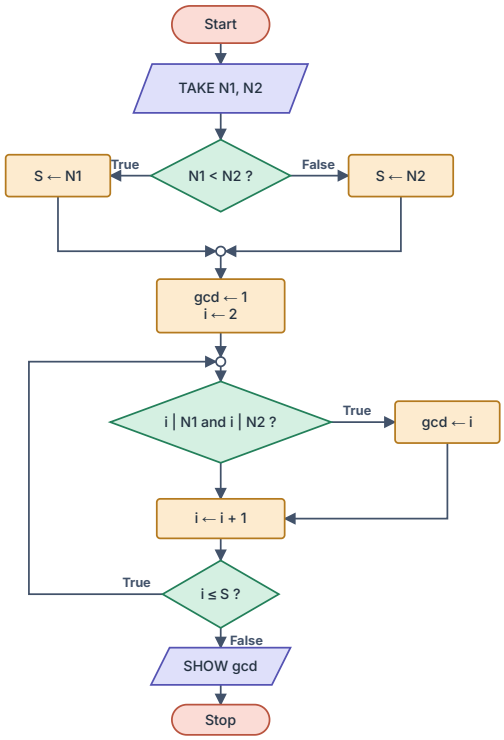
## 1. Greatest common divisor (GCD), trying every number

The GCD of two numbers is the biggest number that divides both. GCD of 12 and 18 is 6. The simple idea: try 2, 3, 4... up to the smaller number, and remember the last one that worked.

**Step 1.** Take N1 and N2.
**Step 2.** if N1 < N2 then
**Step 2.1** S ← N1
else
**Step 2.1** S ← N2
**Step 3.** gcd ← 1, i ← 2
**Step 4.** if i | N1 and i | N2 then
**Step 4.1** gcd ← i
**Step 5.** i ← i + 1
**Step 6.** if i ≤ S then go to step 4
**Step 7.** Show gcd.

**Why gcd starts at 1**

1 divides everything. If no other number works, the answer is 1.




GCD: try every number from 2 up to the smaller one.

```python
N1 = int(input("Enter the 1st number : "))
N2 = int(input("Enter the 2nd number : "))

if N1 < N2:
    S = N1
else:
    S = N2

gcd = 1

for i in range(2, S + 1):
    if N1 % i == 0 and N2 % i == 0:
        gcd = i

print(f"gcd of {N1} and {N2} is: {gcd}")
```

ON THE SCREEN

```
Enter the 1st number : 12
Enter the 2nd number : 18
gcd of 12 and 18 is: 6
```

## 2. GCD, faster: count down and stop

If we start from the top, the **first** match is the biggest. So we can stop right there with `break`. Starting at S and going down to 1 needs `range(S, 0, -1)`.

```python
N1 = int(input("Enter the 1st number : "))
N2 = int(input("Enter the 2nd number : "))

if N1 < N2:
    S = N1
else:
    S = N2

gcd = S

for i in range(S, 0, -1):
    if N1 % i == 0 and N2 % i == 0:
        gcd = i
        break

print(f"gcd of {N1} and {N2} is: {gcd}")
```

ON THE SCREEN

```
Enter the 1st number : 21
Enter the 2nd number : 7
gcd of 21 and 7 is: 7
```

## 3. GCD, the clever way: Euclid’s algorithm

Divide the big number by the small one. Then the small one becomes the new big one, and the remainder becomes the new small one. Repeat until the remainder is 0. The last divisor is the GCD.

| 120 and 50 | 781 and 484 | 1071 and 462 |
|---|---|---|
| 120 = 50·2 + 20 | 781 = 484·1 + 297 | 1071 = 462·2 + 147 |
| 50 = 20·2 + 10 | 484 = 297·1 + 187 | 462 = 147·3 + 21 |
| 20 = **10**·2 + 0 | 297 = 187·1 + 110 | 147 = **21**·7 + 0 |
| **GCD = 10** | 187 = 110·1 + 77 | **GCD = 21** |
| | 110 = 77·1 + 33 | |
| | 77 = 33·2 + 11 | |
| | 33 = **11**·3 + 0 | |
| | **GCD = 11** | |

**Step 1.** Take N1 and N2.

**Step 2.** if N1 < N2 then

**Step 2.1** S ← N1, B ← N2

else

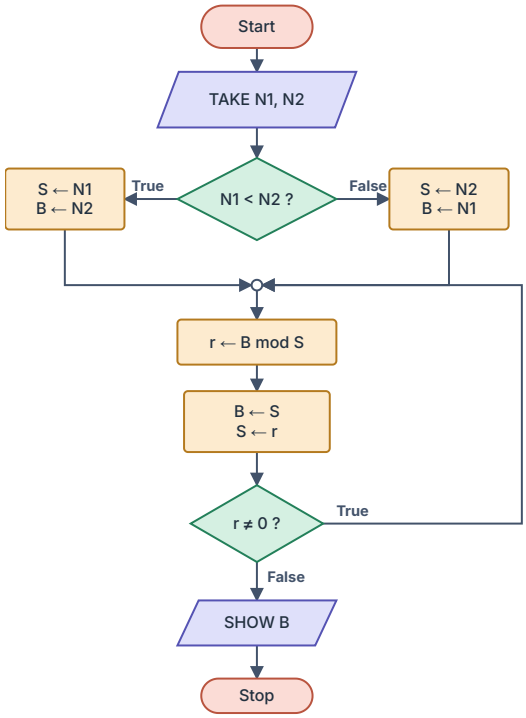
**Step 2.1** S ← N2, B ← N1

**Step 3.** r ← B mod S

**Step 4.** B ← S, S ← r

**Step 5.** if r ≠ 0 then go to step 3

**Step 6.** Show B.




Euclid’s algorithm. We don’t know how many rounds it takes, so it suits while.

```python
N1 = int(input("enter the first number "))
N2 = int(input("enter the second number "))

if N1 < N2:
    s = N1
    B = N2
else:
    s, B = N2, N1

r = s
while r != 0:
    r = B % s
    B, s = s, r

print(f"gcd of {N1} and {N2} is {B}")
```

ON THE SCREEN

```
enter the first number 120
enter the second number 50
gcd of 120 and 50 is 10
```

The same thing with while True and a “loop breaker”

```python
N1 = int(input("enter the first number "))
N2 = int(input("enter the second number "))

if N1 < N2:
    s = N1
    B = N2
else:
    s, B = N2, N1

while True:
    r = B % s
    B, s = s, r
    if r == 0:
        break

print(f"gcd of {N1} and {N2} is {B}")
```

ON THE SCREEN

```
enter the first number 1071
enter the second number 462
gcd of 1071 and 462 is 21
```

## 4. Fibonacci numbers

Start with 1, 1. Each new number is the sum of the two before it: **1, 1, 2, 3, 5, 8, 13, 21, ...** The 7th is 13.

**Step 1.** Take n.
**Step 2.** f1 ← 1, f2 ← 1, i ← 3
**Step 3.** fN ← f1 + f2
**Step 4.** i ← i + 1
**Step 5.** f1 ← f2, f2 ← fN
**Step 6.** if i ≤ n then go to step 3
**Step 7.** Show fN.

**Order is important**

In step 5, f1 must take f2 **before** f2 changes. Reverse them and the old f2 is lost. In Python, `f1, f2 = f2, fn` does both at once, safely.

With for: i is handled for us

```python
n = int(input("enter the index : "))

f1 = f2 = 1

for i in range(3, n + 1):
    fn = f1 + f2
    f1, f2 = f2, fn

print(f"{n}th fibonacci is {fn}")
```

ON THE SCREEN

```
enter the index : 7
7th fibonacci is 13
```

With while: prepare i, change i

```python
n = int(input("enter the index : "))

f1 = f2 = 1
i = 3

while i <= n:
    fn = f1 + f2
    f1, f2 = f2, fn
    i += 1

print(f"{n}th fibonacci is {fn}")
```

ON THE SCREEN

```
enter the index : 7
7th fibonacci is 13
```

**A limit of this code**

For n = 1 or n = 2 the loop never runs, so `fn` is never created and the program crashes. The answer there should be 1. Writing `fn = 1` before the loop fixes it.

## 5. Sum of even Fibonacci numbers below N

For N = 50: the evens are 2, 8 and 34, so the answer is **44**. We don't know how many Fibonacci numbers fit below N, so we use `while True` with a break.

```python
N = int(input("enter your max number : "))

f1 = f2 = 1
s = 0
while True:
    fn = f1 + f2
    if fn > N:
        break
    f1, f2 = f2, fn
    if fn % 2 == 0:
        s += fn

print(f"sum of even fibonaccies is {s}")
```

ON THE SCREEN

```
enter your max number : 50
sum of even fibonaccies is 44
```

**Small print**

The question says “less than N” while the code tests `fn > N`. So if N itself is a Fibonacci number, it gets counted. For strict “less than”, use `fn >= N`.

## 6. Factorial

N! is the product of all the whole numbers from 2 up to N. 5! = 5·4·3·2 = 120. By definition, 0! and 1! are both 1.

**Step 1.** Take N.

**Step 2.** f ← 1, i ← 2

**Step 2.5** if N = 0 or N = 1 then go to step 6

**Step 3.** f ← f * i

**Step 4.** i ← i + 1

**Step 5.** if i ≤ N then go to step 3

**Step 6.** Show f.

```python
N = int(input("enter the number : "))

f = 1
i = 2

while i <= N:
    f *= i
    i += 1

print(f"{N}! = {f}")
```

ON THE SCREEN
```
enter the number : 5
5! = 120
```

```python
N = int(input("enter the number : "))

f = 1

for i in range(2, N + 1):
    f *= i  # f = f * i

print(f"{N}! = {f}")
```

ON THE SCREEN
```
enter the number : 5
5! = 120
```

**Why no check for 0 and 1 in the code?**

The notes first write `if not (N == 0 or N == 1):`, then remove it. For N = 0 or 1, `i <= N` is false from the start, so the loop just never runs and f stays 1. Correct, with no extra line.

## 7. How many digits?

7884 has 4 digits, because it is below $10^4$ but not below $10^3$. So compare N with 10, 100, 1000... until it is smaller.

**Step 1.** Take N.

**Step 2.** i ← 1

**Step 3.** if N < $10^i$ then show i, stop.

**Step 4.** i ← i + 1

**Step 5.** go to step 3

```python
N = int(input("enter the number : "))

i = 1
while True:
    if N < 10 ** i:
        break
    i += 1

print(f"{N} has {i} digits.")
```

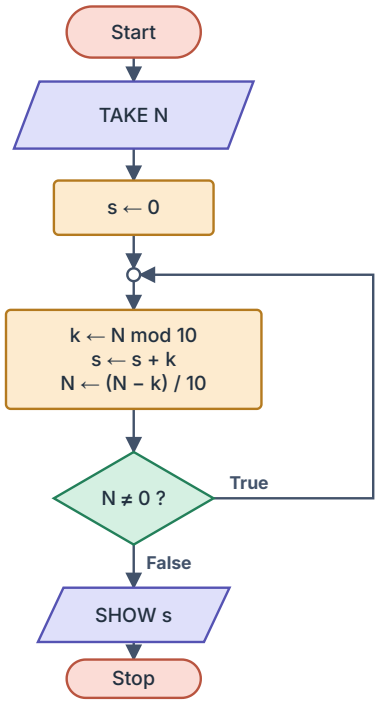
ON THE SCREEN
```
enter the number : 321654321654
321654321654 has 12 digits.
```

## 8. Sum of the digits

For 851: `% 10` peels off the last digit, `// 10` removes it. Repeat until nothing is left.

| N | k = N % 10 | s |
|---|---|---|
| 851 | 1 | 1 |
| 85 | 5 | 6 |
| 8 | 8 | 14 |
| 0 | stop | **14** |

**Step 1.** Take N.
**Step 1.5** s ← 0
**Step 2.** k ← N mod 10
**Step 2.5** s ← s + k
**Step 3.** N ← (N − k) / 10
**Step 4.** if N ≠ 0 then go to step 2
**Step 5.** Show s.




The flowchart the lecturer left as "a task for you".

```python
N = int(input("enter the number : "))

s = 0

while N != 0:
    k = N % 10
    s += k
    N = N // 10

print(f"the sum of digits is {s}")
```

```
ON THE SCREEN
enter the number : 851
the sum of digits is 14
```

**Positive numbers only**

These digit programs assume N is not negative. With a negative number, `//` rounds down and gets stuck at −1, so `while N != 0` would never end. To handle negatives, start with `N = abs(N)`.

**/ versus //**

In the notes, `N = (N - k) / 10` printed 17.0 instead of 17, because `/` always makes a float. `N // 10` keeps it a whole number. `int((N - k) / 10)` also works.

## 9. A decimal fraction in binary

Computers count in base 2, using only 0 and 1. To convert a number between 0 and 1, such as 0.8125: **double it**. If the result is 1 or more, the next binary digit is 1, and you remove that 1. If not, the digit is 0. Repeat until nothing is left.

| N | 2 × N | Digit | N - digit |
|---|---|---|---|
| 0.8125 | 1.625 | **1** | 0.625 |
| 0.625 | 1.25 | **1** | 0.25 |
| 0.25 | 0.5 | **0** | 0.5 |
| 0.5 | 1.0 | **1** | 0 |

Reading the digits from top to bottom: 0.8125 in base 10 is **0.1101** in base 2.

**Step 1.** Take N.
**Step 2.** N ← 2N
**Step 3.** if N ≥ 1 then
  **Step 3.5** digit ← 1
  else
  **Step 3.5** digit ← 0
**Step 4.** Show digit.
**Step 5.** N ← N − digit
**Step 6.** if N ≠ 0 then go to step 2

```python
N = float(input("enter the number : "))

print("0.", end="")

while N != 0:
    N = 2 * N
    if N >= 1:
        digit = 1
    else:
        digit = 0
    print(digit, end="")
    N -= digit

print("")
print("have a nice day...")
```

ON THE SCREEN

```
enter the number : 0.8125
0.1101
have a nice day...
```

`end=""` keeps the digits on one line. By default `end` is `"\n"`, which works like pressing Enter.

## 10. Reject negatives, stop at zero

This is the program at the end of Chapter 5 (lecture of 23.10). In its flowchart, the arrow back from `number < 0` acts as `continue`, and the arrow out at `number = 0` acts as `break`. The main loop itself has no condition, which is why it becomes `while True:`.

The question in the lecture also says to **count how many numbers are positive**, although the lecture's solution only shows the sum. Here is the full version, with the count added:

```python
s = 0
count = 0
while True:
    number = int(input("Enter a number: "))
    if number < 0:
        continue
    if number == 0:
        break
    s += number
    count += 1

print(f"{count} positive numbers, their sum is {s}")
```

ON THE SCREEN

```
Enter a number: 7
Enter a number: -1
Enter a number: 3
Enter a number: 0
2 positive numbers, their sum is 10
```

CHAPTER 8

# How to answer each exam question type

The exercise sheet has five kinds of question. Here is a method for each.

## Type A: “Write all values” (tracing)

Pretend to be the computer. Draw one column per variable and go line by line. Each time a value is assigned, write it down. Do not skip lines, and do not use your intuition.

Example

```
1  t = 0
2  for i in range(1, 4):
3      t = t + i
```

| Line reached | i | t | What happened |
|---|---|---|---|
| 1 | | 0 | t starts at 0 |
| 2 | 1 | | range gives 1 |
| 3 | | 1 | 0 + 1 |
| 2 | 2 | | range gives 2 |
| 3 | | 3 | 1 + 2 |
| 2 | 3 | | range gives 3 |
| 3 | | 6 | 3 + 3. Done: 4 is not included |

**range is not affected by the body**

If you change `i` inside a `for`, the next round still takes the next number from `range`, as if nothing happened. This is the trick in exercise 2.

## Type B: Flowchart → Python

| In the flowchart | In Python |
|---|---|
| Oval (Start, Stop) | Nothing. It’s just the beginning and the end of the file. |
| Parallelogram TAKE x | `x = int(input("…"))` |
| Parallelogram SHOW x | `print(x)` |

| In the flowchart | In Python |
|---|---|
| Rectangle x ← y + 1 | `x = y + 1` |
| x mod 2 | `x % 2` |
| i \| A (i divides A) | `A % i == 0` |
| Diamond with no arrow going back | `if … else …` |
| Diamond at the **top** of a loop | `while condition:` |
| Diamond at the **bottom or middle** of a loop | `while True:` and `if …: break` |
| Double-sided box w = f(c) | `def f(x): …` at the top, and `w = f(c)` here |

**Check which way the arrow leaves**

Python's `while` *continues* on true. If the flowchart leaves the loop on True, like "x ≥ y → SHOW", then flip the question: `while not x >= y:`, or simply `while x < y:`.

## Type C: Problem → algorithm → flowchart → code

1. **Try it by hand** with small numbers. How did you do it?
2. **Name the boxes** you needed (a total, a counter).
3. **Write the steps** in the Step 1, Step 2 style.
4. **Draw the flowchart**: one shape per step, and an arrow back for every "go to".
5. **Write the code**, then test with your hand example.

| If the question says | Think of |
|---|---|
| "sum of…", "total" | accumulator: `s = 0`, `s += x` |
| "how many…", "count" | counter: `c = 0`, `c += 1` |
| "average" | both: total ÷ count. Check the count isn't 0 |
| "divisible by", "even", "odd" | `%` |
| "100 numbers", "N numbers" | `for i in range(N):` |
| "until…", "when zero is entered" | `while True:` and `break` |
| different results for different ranges | `if / elif / else` |
| "digits of a number" | `% 10` and `// 10` |

## Type D: for → while

A `for i in range(a, b, c):` is the same as these three pieces. Move them to the right places.

Translation pattern (for a positive step c)

```
i = a             # prepare, before the loop
while i < b:      # test (the end is not included)
    ...           # the body
    i += c        # change, as the LAST line of the body
```

**Nested loops**

The inner counter must be reset **inside** the outer loop, so it restarts every time.

**Two cases where the pattern needs care**

**If the body uses `continue`:** `continue` would jump over `i += c` and the loop might never end. Do the `i += c` before the `continue` too.

**If the body changes i:** a `for` ignores that change, as exercise 2 showed, but a `while` does not. Count with a separate variable, so the body's changes to i can't affect the counting.

## Type E: Fill in the blanks

Work out what the finished program should do, write down the list of values you need, then choose the range. Test the first and last value. "2 to 12, even" becomes `range(2, 13, 2)`.

## Before you hand in: a checklist

- Every `if`, `elif`, `else`, `for`, `while` and `def` line ends with `:`.
- Lines inside are indented. Lines after are not.
- `==` when asking, `=` when storing.
- `**` for powers, never `^`.
- Every `input` that needs a number is wrapped in `int(…)` or `float(…)`.
- Totals and counters are set up **before** the loop, not inside it.
- Every `while` changes something, so it can end.
- Division has brackets around the whole top and the whole bottom.
- In flowcharts: oval, parallelogram, rectangle, diamond are used correctly, and each diamond has two labelled exits.

CHAPTER 9

# The exercise sheet, fully solved

Try each one yourself first and cover the answer. Every program here has been run, and the output shown is real.

**EXERCISE 1** Write a Python program for the flowchart.

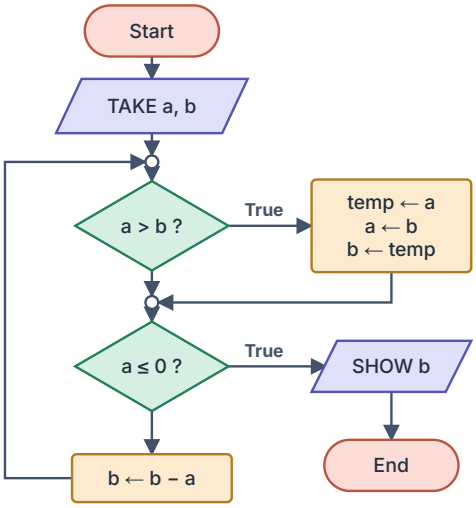
```python
a = int(input("a = "))
b = int(input("b = "))

while True:
    if a > b:
        temp = a
        a = b
        b = temp
    if a <= 0:
        break
    b = b - a

print(b)
```

ON THE SCREEN

```
a = 12
b = 18
6
```




**What it is:** a GCD. It keeps subtracting the smaller number from the bigger one. When a reaches 0, b is the GCD. The decision is in the middle of the loop, so `while True` and `break` fit best.

**EXERCISE 2** Write all values, in assigning order, of a, i and t.

```python
i = t = 0
for i in range(2, 8, 3):
    t += i
    i = 3 * t + 1
    a = 2 * i

t = a - i
```

| Step | Assignment | Why |
|---|---|---|
| 1 | i = 0, t = 0 | first line |
| 2 | i = 2 | range(2, 8, 3) gives 2, then 5 |
| 3 | t = 2 | 0 + 2 |
| 4 | i = 7 | 3·2 + 1 |
| 5 | a = 14 | 2·7 |
| 6 | i = 5 | next from range. The 7 doesn't matter! |
| 7 | t = 7 | 2 + 5 |
| 8 | i = 22 | 3·7 + 1 |
| 9 | a = 44 | 2·22 |
| 10 | t = 22 | after the loop: 44 − 22 |

**EXERCISE 3** Re-write using while: `for i in range(2, 25, 3):` with inner `for j in range(1, 2 * i):` printing i and j.

```
i = 2
while i < 25:
    j = 1
    while j < 2 * i:
        print("i is ", i, " and   j is ", j)
        j += 1
    i += 3
```

ON THE SCREEN

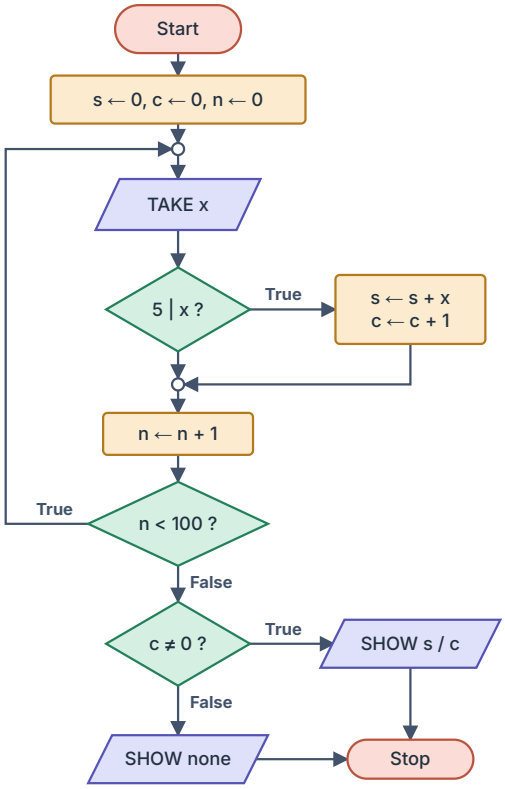
```
i is  2  and   j is  1
i is  2  and   j is  2
i is  2  and   j is  3
i is  5  and   j is  1
… (188 more lines)
```

Both versions print the same 192 lines. `j = 1` must be inside the outer loop.

**How it runs:** for i = 2, j takes 1, 2, 3. Then i becomes 5, j goes back to 1 and runs up to 9. `j += 1` is inside the inner loop, and `i += 3` is inside only the outer loop.

**EXERCISE 4** Write an algorithm to calculate the AVERAGE of numbers divisible by 5, within 100 integers given by the user. Then output the average. Draw the flowchart. Write the Python code.

**Step 1.** s ← 0, c ← 0, n ← 0
**Step 2.** Take x.
**Step 3.** if 5 | x then
  **Step 3.1** s ← s + x, c ← c + 1
**Step 4.** n ← n + 1
**Step 5.** if n < 100 then go to step 2
**Step 6.** if c ≠ 0 then
  **Step 6.1** Show s / c
  else
  **Step 6.1** Show “none”




```python
s = 0
c = 0
for n in range(100):
    x = int(input("enter a number: "))
    if x % 5 == 0:
        s += x
        c += 1

if c != 0:
    print(s / c)
else:
    print("no number is divisible by 5")
```

**ON THE SCREEN**

```
7.5
```

Tested with 100 numbers (5, 10, 3, then 1s). Only the last line is shown.

**Dividing by zero**

If no number is divisible by 5, c stays 0, and `s / c` would crash. That check is an easy extra point.

**EXERCISE 5** Complete the code to write a given integer in reverse (123456 → 654321). Using numbersOfDigits is optional.

```python
number = int(input("enter an integer "))

while True:
    digit = number % 10
    print(digit, end="")
    number = number // 10
    if number == 0:
        break
```

**ON THE SCREEN**

```
enter an integer 123456
654321
```

Same peel-the-last-digit idea as the sum of digits, for a number that isn't negative. Printing each digit on the same line keeps zeros too: 1200 gives 0021.

**EXERCISE 6** Take 100 integers from the user. Count how many are EVEN. Output the result. Draw the flowchart. Write the code.

**Step 1.** c ← 0, n ← 0

**Step 2.** Take x.

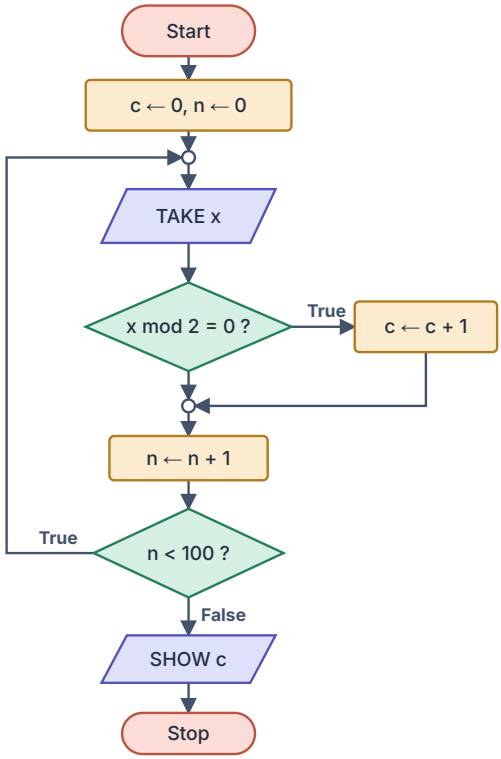
**Step 3.** if x mod 2 = 0 then

**Step 3.1** c ← c + 1

**Step 4.** n ← n + 1

**Step 5.** if n < 100 then go to step 2

**Step 6.** Show c.




```python
c = 0
for n in range(100):
    x = int(input("enter a number: "))
    if x % 2 == 0:
        c += 1

print(c)
```

**ON THE SCREEN**

```
2
```

Tested with 100 numbers, two of them even. Only the last line is shown.

**EXERCISE 7** Calculate f(x) for x from the user: $x^3$ if x > 0; −x·cos(x) if x is in [−2, 0]; 1/4 + $x^2$ if x < −2.

```
import math as m

x = float(input("x = "))

if x > 0:
    f = x ** 3
elif x >= -2:
    f = -x * m.cos(x)
else:
    f = 1 / 4 + x ** 2

print(f)
```

ON THE SCREEN

```
x = 2
8.0
```

Having passed the first test, x is not above 0, so `x >= -2` is enough for the range [−2, 0].

**EXERCISE 8** Fill in the blanks so that t is the sum of even numbers from 2 to 12, except 8.

```
t = 0
for i in range(2, 13, 2):
    if i == 8:
        continue
    t += i
print(t)
```

ON THE SCREEN

```
34
```

The blanks are: `2, 13, 2` and `i == 8` and `continue`. 2 + 4 + 6 + 10 + 12 = 34.

**EXERCISE 9** Write all values, in assigning order, of i and t.

```
t = -4
for i in range(1, 7, 2):
    if i + t == 1:
        continue
    t = t + 3 * i - 1
```

Round i = 3 is skipped, because 3 + (−2) = 1, so t keeps its value there.

| Step | Assigned | Check |
|---|---|---|
| 1 | t = −4 | |
| 2 | i = 1 | 1−4 ≠ 1 |
| 3 | t = −2 | −4+3−1 |
| 4 | i = 3 | 3−2 = 1, skip |
| 5 | i = 5 | 5−2 ≠ 1 |
| 6 | t = 12 | −2+15−1 |

**EXERCISE 10** (a) Write Python code for the flowchart. (b) What do we see on the screen when x = 5 and y = 17?

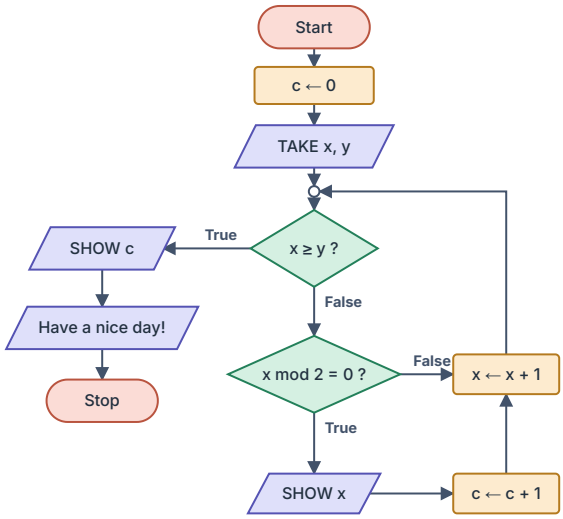
```
c = 0
x = int(input("x = "))
y = int(input("y = "))

while x < y:
    if x % 2 == 0:
        print(x)
        c = c + 1
    x = x + 1

print(c)
print("Have a nice day!")
```

ON THE SCREEN

```
x = 5
y = 17
6
8
10
12
14
16
6
Have a nice day!
```




**(b) Answer:** 6, 8, 10, 12, 14, 16, then 6, then “Have a nice day!”, each on its own line. The first two lines in the box are just the typed inputs.

**In words:** it prints every even number from x up to (not including) y, then how many there were.

**EXERCISE 11** Write all assigned values of c and ns when s = "Py+hon", oc = "+", nc = "t".

```
1  s = input("enter a text : ")
2  oc = input("enter old character : ")
3  nc = input("enter new character : ")
4
5  ns = ""
6  for c in s:
7      if c == oc:
8          ns = ns + nc
9      else:
10         ns = ns + c
11
12 print(s)
13 print(ns)
```

ON THE SCREEN

```
enter a text : Py+hon
enter old character : +
enter new character : t
Py+hon
Python
```

| Assigned c | Assigned ns |
|---|---|
| (start) | "" |
| P | P |
| y | Py |
| + | Pyt |
| h | Pyth |
| o | Pytho |
| n | Python |

`+` joins strings together. This program replaces one character with another.

**EXERCISE 12** BMI: write an algorithm that takes weight and height and responds with the risk level. Write the Python program too.

| Classification | BMI | Risk |
|---|---|---|
| Underweight | < 18.5 | Increased |
| Normal Weight | 18.5 – 24.9 | Least |
| Overweight | 25.0 – 29.9 | Increased |
| Obese | 30.0 and above | High |

**Check the formula**

The sheet prints BMI = $wh^2$, with the unit $kg/m^2$. A unit of $kg/m^2$ means the correct formula is **$w / h^2$**, with h in metres, so we use that. It's worth asking your lecturer about it.

**Step 1.** Take w and h.

**Step 2.** BMI ← w / h²

**Step 3.** if BMI < 18.5 then risk ← Increased
else if BMI < 25 then risk ← Least
else if BMI < 30 then risk ← Increased
else risk ← High

**Step 4.** Show risk.

```
w = float(input("weight (kg): "))
h = float(input("height (m): "))

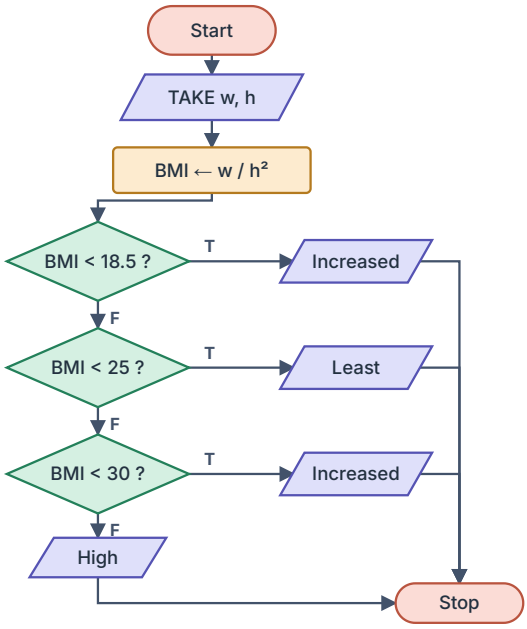
bmi = w / h ** 2

if bmi < 18.5:
    risk = "Increased"
elif bmi < 25:
    risk = "Least"
elif bmi < 30:
    risk = "Increased"
else:
    risk = "High"

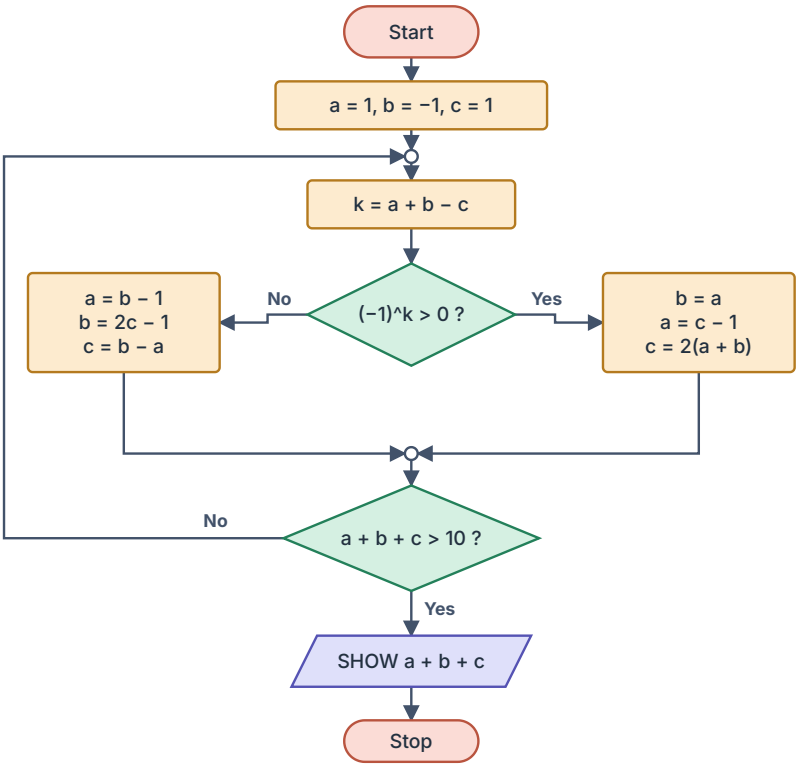
print(risk)
```

ON THE SCREEN

```
weight (kg): 70
height (m): 1.75
Least
```



**EXERCISE 13** Write a Python program for the flowchart.




```python
a = 1
b = -1
c = 1

while True:
    k = a + b - c
    if (-1) ** k > 0:
        b = a
        a = c - 1
        c = 2 * (a + b)
    else:
        a = b - 1
        b = 2 * c - 1
        c = b - a
    if a + b + c > 10:
        break

print(a + b + c)
```

ON THE SCREEN

```
12
```

| Round | k | (-1)^k > 0? | a | b | c | a+b+c |
|---|---|---|---|---|---|---|
| 1 | -1 | No | -2 | 1 | 3 | 2 |
| 2 | -4 | Yes | 2 | -2 | 0 | 0 |
| 3 | 0 | Yes | -1 | 2 | 2 | 3 |
| 4 | -1 | No | 1 | 3 | 2 | 6 |
| 5 | 2 | Yes | 1 | 1 | 4 | 6 |
| 6 | -2 | Yes | 3 | 1 | 8 | **12** |

**Separate lines, in order**

Each box uses the values just changed above it. Don't combine these into `a, b, c = …`. That would use the old values and give a wrong result.

**EXERCISE 14** Taxi fare: C(x) = 120 TL if x ≤ 3500 metres, otherwise 7 TL for each 200 metres. Write the algorithm and the Python program. Round down when counting 200 metres. (The driver can enter the distance in km. Take precautions!)

**Step 1.** Take x and its unit.

**Step 2.** if the unit is km then

**Step 2.1** x ← x · 1000

**Step 3.** if x ≤ 3500 then

**Step 3.1** C ← 120

else

**Step 3.1** C ← 7 · (x / 200, rounded down)

**Step 4.** Show C.

```python
x = float(input("distance: "))
unit = input("m or km? ")

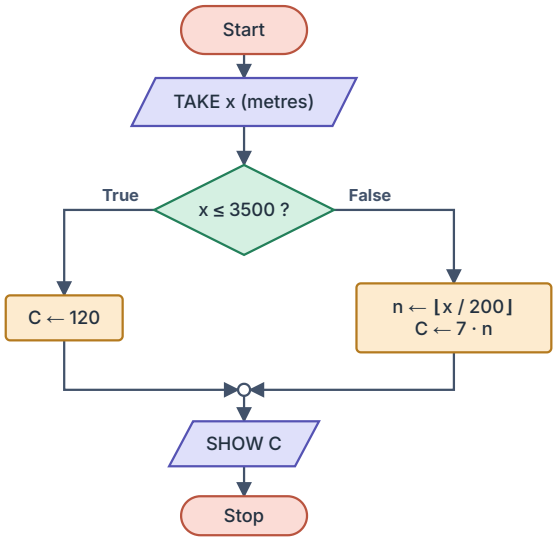
if unit == "km":
    x = x * 1000

if x <= 3500:
    fare = 120
else:
    fare = 7 * int(x / 200)

print(f"Fare: {fare} TL")
```

**ON THE SCREEN**

```
distance: 3.8
m or km? km
Fare: 133 TL
```




| Distance | x in metres | x / 200, rounded down | Fare |
|---|---|---|---|
| 3500 m | 3500 | not used | 120 TL |
| 3700 m | 3700 | 18 | 126 TL |
| 3799 m | 3799 | 18 | 126 TL |
| 3.8 km | 3800 | 19 | 133 TL |

**The "precaution":** 3.8 in km must become 3800 metres before anything else. `int(…)` does the rounding down. Asking for the unit is the clearest way to handle it.

> **An odd result to know about**
>
> Taken literally, the formula makes 3600 m cost 7 × 18 = 126 TL, while 3500 m costs 120 TL, and distances from 3501 to 3599 m cost 119 TL, which is *less* than 3500 m. The sheet may have meant something else, but answer the formula as printed.

**EXERCISE 15** Write a Python program for the flowchart.

```
A = int(input("A = "))
B = int(input("B = "))

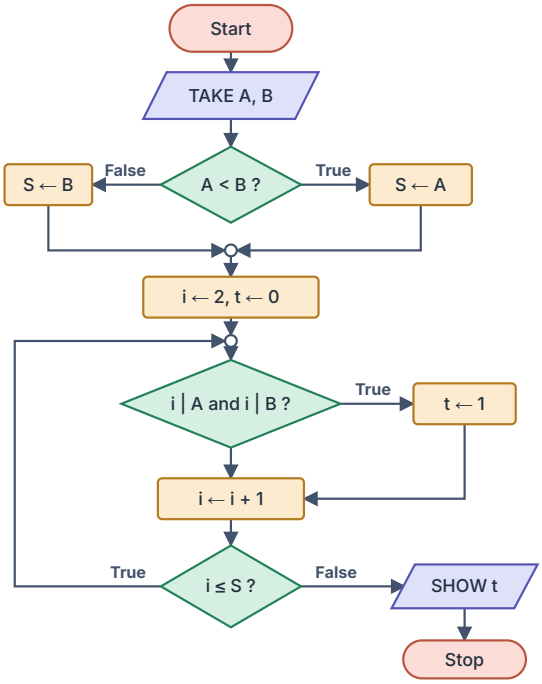
if A < B:
    S = A
else:
    S = B

i = 2
t = 0
while True:
    if A % i == 0 and B % i == 0:
        t = 1
    i = i + 1
    if not i <= S:
        break

print(t)
```

ON THE SCREEN

```
A = 12
B = 18
1
```




**What it means:** t is 1 if A and B share a divisor bigger than 1. It is 0 if they don't: (12, 18) gives 1, (7, 13) gives 0. `i | A` is written `A % i == 0`.

**EXERCISE 16** Write the Python program of the flowchart, where f(x) = x² − 9.

```python
def f(x):
    return x ** 2 - 9

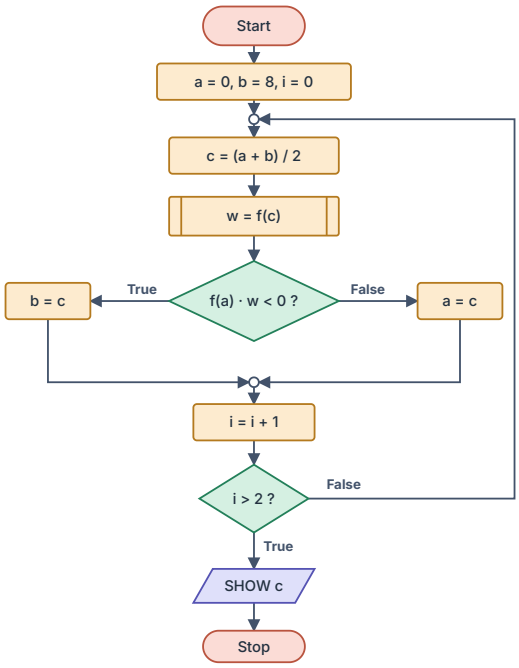

a = 0
b = 8
i = 0

while True:
    c = (a + b) / 2
    w = f(c)
    if f(a) * w < 0:
        b = c
    else:
        a = c
    i = i + 1
    if i > 2:
        break

print(c)
```

**ON THE SCREEN**

```
3.0
```




| i | a | b | c | w |
|---|---|---|---|---|
| 0→1 | 0 | 8→4 | 4 | 7 |
| 1→2 | 0→2 | 4 | 2 | -5 |
| 2→3 | 2→3 | 4 | 3 | 0 |

**What it is:** a search for where f(x) = 0, halving the interval each round. It finds x = 3, which is correct, since 3² − 9 = 0.

CHAPTER 10

# Cheat sheet

Two pages with everything on them. Print these and keep them close while you revise.

## Key definitions

- **Algorithm:** a method of solution expressed as a sequence of finite steps.
- **Its 4 characteristics:** input, output, definiteness, finiteness.
- **3 control structures:** sequence, branching (selection), loop (repetition).
- **Flowchart:** a blueprint of a solution using symbols. First design: 1945, John von Neumann.
- **Function:** a block of code that only runs when called.

## Flowchart symbols

| | |
|---|---|
| **Oval** | start and end |
| **Parallelogram** | input and output (TAKE, SHOW) |
| **Rectangle** | processing (x ← y + 1) |
| **Diamond** | decision (true or false) |
| **Arrow** | flow of logic |
| **Double-sided box** | predefined process, function |

## Operators

| | |
|---|---|
| `/` | divide, always a float: 10/2 is 5.0 |
| `//` | integer part: 851//10 is 85 |
| `%` | remainder: 851%10 is 1 |
| `**` | power: 2**3 is 8. **Never ^** |
| `== !=` | equal, not equal |
| `< > <= >=` | comparisons |
| `and or not` | both, at least one, flip |
| `+= -= *= /=` | x += 1 means x = x + 1 |
| `a, b = b, a` | swap |

## Types and conversion

```
int 7    float 3.14    str "hi"    bool True
int(25.5) → 25    float("3") → 3.0
n = int(input("how many? "))
x = float(input("value? "))
```

## Printing

```
print("sum:", s)        # comma adds a space
print(f"sum: {s}")      # f-string
print("a\tb")           # \t tab, \n new line
print(x, end="")        # stay on the same line
```

## Math package

```
import math as m
m.pi   m.sqrt(x)   m.pow(a, b)   m.cos(x)
abs(-3) → 3     1j → the complex i
```

## Branching

```
if avg >= 90:
    grade = "AA"
elif avg >= 85:        # first true one wins
    grade = "BA"
else:                  # everything left
    grade = "FF"
```

## range

| | |
|---|---|
| `range(6)` | 0 1 2 3 4 5 |
| `range(2, 6)` | 2 3 4 5 |
| `range(2, 22, 5)` | 2 7 12 17 |
| `range(5, 0, -1)` | 5 4 3 2 1 |
| `range(2, S + 1)` | 2 3 ... S (end is excluded) |

## Loops

```
for i in range(N):      # N rounds
    ...

i = 1                   # 1 prepare
while i <= 10:          # 2 test
    ...
    i += 1              # 3 change, or it never
ends

while True:             # check in the middle
    ...
    if done:
        break           # leave the loop

continue                # skip to the next round
```

## Standard patterns

```
s = 0                    # total
c = 0                    # count
f = 1                    # product (starts at 1!)
for ... :
    s += x
    c += 1
    f *= x
if c != 0:
    print(s / c)         # average

n % 2 == 0               # n is even
n % 5 == 0               # 5 divides n
n % 10                   # last digit
n // 10                  # remove last digit
if a < b: S = a          # smaller of two
```

## Functions

```
def factorial(n):        # define
    f = 1
    for i in range(1, n + 1):
        f *= i
    return f             # send result back

print(factorial(5))      # call → 120
```

## The classics, in one line each

| | |
|---|---|
| **GCD simple** | for i in 2..S: if i divides both, gcd = i |
| **GCD down** | range(S, 0, -1), first match, break |
| **Euclid** | r = B % s; B, s = s, r until r is 0; answer B |
| **Fibonacci** | fn = f1 + f2; f1, f2 = f2, fn |
| **Factorial** | f = 1; f *= i for i in 2..N |
| **Digit count** | smallest i with N < 10**i |
| **Digit sum** | s += N % 10; N = N // 10 until 0 |
| **Fraction to binary** | double; digit is 1 if ≥ 1; subtract digit |
| **Series** | summ += x**k / factorial(k) |

## Flowchart to Python

| | |
|---|---|
| TAKE x | `x = int(input())` |
| SHOW x | `print(x)` |
| x mod 2 | `x % 2` |
| i \| A | `A % i == 0` |
| Diamond, no return arrow | `if / else` |
| Diamond on top of loop | `while cond:` |
| Diamond at bottom or middle | `while True` + `break` |

## Top exam mistakes

- Writing `^` instead of `**`
- `=` where `==` is needed
- Missing `:` or wrong indentation
- Forgetting that range excludes the end
- Forgetting `i += 1` in a while
- Forgetting `int(...)` around input
- Bad brackets: use `(-b + r) / (2 * a)`
- Wrong update order: f1 before f2
- Setting the total to 0 *inside* the loop
- Changing i in a for and expecting it to matter

CHAPTER 11

# Test yourself

No peeking. Circle your answers, then check Chapter 12. The web page has the same questions with instant feedback.

## Part 1: Multiple choice

**1 What is an algorithm?**

**A** A type of computer
**B** A programming language
**C** A drawing with arrows
**D** A method of solution expressed as a sequence of finite steps

**2 Which is NOT one of the four characteristics of an algorithm?**

**A** Speed
**B** Finiteness
**C** Output
**D** Definiteness

**3 "Each instruction must be clear and unambiguous." Which characteristic is this?**

**A** Finiteness
**B** Output
**C** Definiteness
**D** Input

**4 What are the three control structures?**

**A** Input, process, output
**B** Sequence, branching, loop
**C** Start, middle, stop
**D** for, while, if

**5 Which symbol is used for TAKE and SHOW?**

**A** Oval
**B** Diamond
**C** Rectangle
**D** Parallelogram

**6 Which symbol is used for a decision?**

**A** Rectangle
**B** Parallelogram
**C** Diamond
**D** Oval

**7 What does a rectangle with an extra line on each side represent?**

**A** An input
**B** A comment
**C** The end of the program
**D** A predefined process (function)

**8** **According to the slides, who designed the first flowchart, and when?**

**A** Alan Turing, 1936
**B** Ada Lovelace, 1843
**C** John von Neumann, 1945
**D** Guido van Rossum, 1991

---

**9** **Which is a valid variable name?**

**A** not
**B** midterm_1
**C** 1st_midterm
**D** my-name

---

**10** **Are age and Age the same variable?**

**A** Yes
**B** Only for numbers
**C** No, Python is case-sensitive
**D** Only inside a loop

---

**11** **What type does input() give back?**

**A** int
**B** str (text)
**C** bool
**D** float

---

**12** **How do you write r squared in Python?**

**A** r^2
**B** $r^2$
**C** r**2
**D** r*2

---

**13** **What does print(7 % 3) show?**

```
1  print(7 % 3)
```

**A** 2.33
**B** 1
**C** 2
**D** 4

---

**14** **What is shown?**

```
1  print(851 // 10)
```

**A** 85.1
**B** 86
**C** 85
**D** 1

---

**15** **What is shown?**

```
1  print(10 / 2)
```

**A** 5
**B** 10
**C** 2
**D** 5.0

---

**16** What is shown?

```
print(2 + 3 * 4)
```

**A** 24
**B** 20
**C** 14
**D** 9

---

**17** What is shown?

```
a = 5
b = 9
a, b = b, a
print(a, b)
```

**A** 5 9
**B** 9 5
**C** 5 5
**D** 9 9

---

**18** What is shown?

```
a = 5
b = 9
a = b
b = a
print(a, b)
```

**A** 9 5
**B** 9 9
**C** 5 9
**D** 5 5

---

**19** What is shown?

```
x = 4
x += 3
x *= 2
print(x)
```

**A** 7
**B** 14
**C** 9
**D** 11

---

**20** Which line asks whether x equals 3?

**A** if x == 3:
**B** if x := 3
**C** if x = 3:
**D** if x equals 3:

---

**21 What is shown?**

```
a = 7
if a % 2 == 0:
    print("even")
else:
    print("odd")
```

**A** even
**B** nothing
**C** even and odd
**D** odd

---

**22 What is shown?**

```
x = 85
if x >= 70:
    print("C")
elif x >= 80:
    print("B")
else:
    print("F")
```

**A** B
**B** C and B
**C** C
**D** F

---

**23 What is shown?**

```
x = 3
print(x > 1 and x > 5, x > 1 or x > 5)
```

**A** True True
**B** False True
**C** False False
**D** True False

---

**24 What is the same as not (N == 0 or N == 1)?**

**A** N == 0 and N == 1
**B** N != 0 and N != 1
**C** N > 1
**D** N != 0 or N != 1

---

**25 Which numbers does range(4) give?**

**A** 4
**B** 0, 1, 2, 3, 4
**C** 0, 1, 2, 3
**D** 1, 2, 3, 4

---

**26 Which numbers does range(2, 22, 5) give?**

**A** 2, 7, 12, 17, 22
**B** 5, 10, 15, 20
**C** 2, 7, 12, 17
**D** 2, 22, 5

---

**27** **Which range gives 5, 4, 3, 2, 1?**

**A** range(1, 5)
**B** range(5, 0)
**C** range(5, 1, -1)
**D** range(5, 0, -1)

---

**28** **Which range gives 2, 3, ..., S?**

**A** range(1, S)
**B** range(2, S)
**C** range(2, S + 1)
**D** range(S)

---

**29** **What is shown?**

```
for t in "abc":
    print(t, end="")
```

**A** abc
**B** error
**C** t t t
**D** a b c

---

**30** **What is shown?**

```
t = 0
for i in range(1, 4):
    t += i
print(t)
```

**A** 10
**B** 6
**C** 4
**D** 3

---

**31** **What is shown?**

```
for i in range(5):
    if i == 2:
        continue
    print(i, end=" ")
```

**A** 0 1 2
**B** 0 1 3 4
**C** 0 1
**D** 2

---

**32** **What is shown?**

```
for i in range(5):
    if i == 2:
        break
    print(i, end=" ")
```

**A** 0 1 2
**B** 2
**C** 0 1
**D** 0 1 3 4

---

**33 What is wrong with this code?**

```
i = 1
while i <= 3:
    print(i)
```

**A** while needs a range
**B** It prints only once
**C** It never ends, because i never changes
**D** Nothing

---

**34 What is shown?**

```
i = 10
while i > 1:
    i = i // 2
print(i)
```

**A** 1
**B** 5
**C** 0
**D** 2

---

**35 What is the starting value of a running product, like factorial?**

**A** Any value
**B** 1
**C** 0
**D** N

---

**36 Which keyword creates a function?**

**A** return
**B** call
**C** function
**D** def

---

**37 When does the code inside a function run?**

**A** Only when it is called
**B** At the end of the program
**C** Never
**D** As soon as Python reads it

---

**38 What is shown?**

```
def d(x):
    return 2 * x

print(d(d(3)))
```

**A** 12
**B** 3
**C** 9
**D** 6

---

**39 What does return do?**

**A** Prints a value
**B** Asks for input
**C** Sends a result back to where the function was called
**D** Restarts the function

---

**40** Using Euclid's algorithm, what is the GCD of 120 and 50?

**A** 5
**B** 50
**C** 20
**D** 10

---

**41** What is the 7th Fibonacci number?

**A** 8
**B** 13
**C** 11
**D** 21

---

**42** What is shown?

```
N = 851
s = 0
while N != 0:
    s += N % 10
    N = N // 10
print(s)
```

**A** 15
**B** 85
**C** 851
**D** 14

---

**43** What is 0.8125 in binary?

**A** 0.1101
**B** 0.8125
**C** 0.1011
**D** 0.0101

---

**44** Why can GCD with a countdown use break?

**A** for loops require it
**B** break makes it more accurate
**C** The first match when counting down is the biggest
**D** Otherwise it gives an error

---

## Part 2: Written questions

These are like the real exam. Use the space, or a sheet of paper.

**45** **Trace it.** Write all values, in assigning order, of i and s.

```
s = 1
for i in range(1, 8, 3):
    s = s * 2 + i
```

**46** **Trace it.** What is on the screen?

```
x = 3
c = 0
while x < 10:
    if x % 2 == 1:
        c += 1
        print(x)
    x = x + 2
print(c)
```

**47** **Fill in the blanks** so the program adds the odd numbers from 1 to 9, except 5.

```
t = 0
for i in range(____):
    if ____:
        ____
    t += i
print(t)
```

**48** **Rewrite using while.**

```
for k in range(10, 0, -2):
    print(k)
```

**49** **Algorithm and code.** Take N numbers from the user and show the biggest. Write the steps, draw the flowchart, write the Python.

**50** **Write a program.** A shop charges 50 TL for up to 10 items and 4 TL per item above that (so 12 items cost 50 + 8). Take the number of items and show the price.

**51** **Write a function** `count_digits(n)` that returns how many digits a positive integer has, using `//` in a loop. Then print the result for a number from the user.

CHAPTER 12

# Answers

Got one wrong? Good: read why, then go back to that chapter. Mistakes here cost you nothing.

## Part 1: Multiple choice

| | | |
|---|---|---|
| 1 | D | This is the slide's definition, word for word. |
| 2 | A | The four are input, output, definiteness and finiteness. |
| 3 | C | Definite means there is only one way to understand each step. |
| 4 | B | Sequence, branching (selection) and loop (repetition). |
| 5 | D | Parallelograms mean input and output. |
| 6 | C | A diamond has a question and at least two exits. |
| 7 | D | It represents a group of steps defined elsewhere, like w = f(c). |
| 8 | C | The slides give John von Neumann, 1945. |
| 9 | B | 1st starts with a digit, my-name has a dash, not is a keyword. |
| 10 | C | Capital letters matter in names. |
| 11 | B | Always text, which is why we wrap it in int() or float(). |
| 12 | C | ^ is a different operation. The lecturer warns against it. |
| 13 | B | % gives the remainder: 7 = 3·2 + 1. |
| 14 | C | // keeps only the integer part. |
| 15 | D | / always gives a float. |
| 16 | C | Multiplication goes first: 3·4 = 12, then 2 + 12. |
| 17 | B | Both assignments happen at once, so this swaps them. |
| 18 | B | After a = b, the 5 is gone. Then b = a keeps 9. Order matters! |
| 19 | B | 4 + 3 = 7, then 7 · 2 = 14. |
| 20 | A | Two equals signs ask. One equals sign stores. |
| 21 | D | 7 % 2 is 1, so the condition is false. |
| 22 | C | Python stops at the first true condition. Check from highest to lowest! |
| 23 | B | and needs both sides. or needs only one. |

24 **B** The slides point this out. “Not (A or B)” means “not A and not B”.

25 **C** It starts at 0, with 4 items, and doesn’t include 4.

26 **C** Start at 2, add 5, stop before 22.

27 **D** The end is not included, so stop at 0.

28 **C** To include S, end at S + 1.

29 **A** A string gives one letter each round. end="" keeps them on one line.

30 **B** 1 + 2 + 3 = 6. The 4 isn’t included.

31 **B** continue skips only round 2.

32 **C** break leaves the whole loop.

33 **C** This is an infinite loop. i += 1 is missing.

34 **A** 10 → 5 → 2 → 1, then the condition is false.

35 **B** Anything times 0 stays 0.

36 **D** def means define.

37 **A** Slide: “a block of code which only runs when it is called”.

38 **A** d(3) = 6, then d(6) = 12.

39 **C** It hands back the result. print only shows it.

40 **D** 120 = 50·2 + 20, 50 = 20·2 + 10, 20 = 10·2 + 0.

41 **B** 1, 1, 2, 3, 5, 8, 13.

42 **D** 1 + 5 + 8 = 14.

43 **A** Doubling gives digits 1, 1, 0, 1.

44 **C** Once we find a match from the top, nothing later can beat it.

## Part 2: Written questions

45 s = 1, i = 1, s = 3, i = 4, s = 10, i = 7, s = 27. (range gives 1, 4, 7.)

46 3, 5, 7, 9, then 4 (each on its own line). x starts odd and goes up by 2, so all four are odd.

47 `range(1, 10, 2)`, `if i == 5:`, `continue`. Result: 1 + 3 + 7 + 9 = 20.

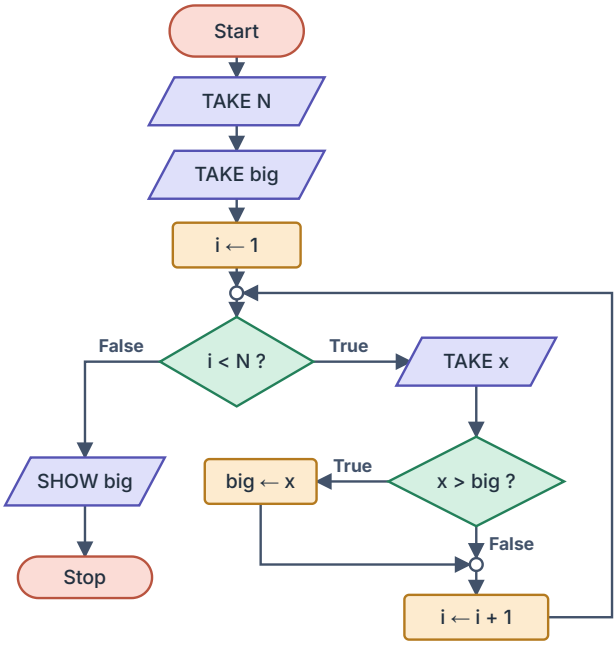
48 Prints 10, 8, 6, 4, 2.

```
k = 10
while k > 0:
    print(k)
    k -= 2
```

ON THE SCREEN

```
10
8
6
4
2
```

**49** Steps: take N (at least 1). Take the first number as big. Repeat N−1 times: take x, and if x > big then big ← x. Show big.




```
N = int(input("how many? "))
big = float(input("number: "))
for i in range(N - 1):
    x = float(input("number: "))
    if x > big:
        big = x
print(big)
```

ON THE SCREEN

```
how many? 3
number: 4
number: 9
number: 2
9.0
```

**50** For 12 items, 58 TL.

```
n = int(input("items: "))
if n <= 10:
    p = 50
else:
    p = 50 + 4 * (n - 10)
print(p)
```

```
ON THE SCREEN
items: 12
58
```

**51**

```
def count_digits(n):
    c = 0
    while n != 0:
        n = n // 10
        c += 1
    return c


number = int(input("number: "))
print(count_digits(number))
```

```
ON THE SCREEN
number: 7884
4
```

## How did you do on Part 1?

| Score | What to do next |
|---|---|
| **39–44** | Excellent. Move on to solving the exercise sheet with no help. |
| **30–38** | Good. Review the topics you missed, then retake the quiz. |
| **0–29** | No problem. Reread Chapters 3 to 5 and trace every code box by hand. |